

Troubleshooting w Kubernetes

Logi, eventy, opis obiektów i pierwszy workflow diagnostyczny

```
kubectl get pods  
kubectl describe pod <pod-name>
```



Cel modułu

- umieć szybko sprawdzić stan aplikacji
- wiedzieć, gdzie Kubernetes zapisuje informacje o problemie
- czytać logi aplikacji i eventy klastra
- odróżnić problem aplikacji od problemu konfiguracji
- zbudować prosty workflow diagnostyczny dla QA



Od czego zaczynamy?

1
status

2
opis

3
logi

4
eventy

5
test

Najpierw sprawdzamy objaw, potem szukamy przyczyny w szczegółach i logach.



Pierwsze spojrzenie: get

```
kubectl get pods  
kubectl get pods -o wide  
kubectl get all
```

czy Pod istnieje?

jaki ma status?

czy ma przypisany node i IP?

czy Deployment utworzył oczekiwaną liczbę Podów?



Typowe statusy Podów

Running

kontener działa, ale aplikacja nadal może być niedostępna

Pending

Pod czeka na zasoby, scheduler albo wolumen

**ImagePullBackO
ff**

problem z pobraniem obrazu kontenera

**CrashLoopBack
Off**

aplikacja startuje i zaraz się kończy



describe pod

```
kubectl describe pod <pod-name>
```

szczegóły Poda i kontenerów

zmienne środowiskowe, wolumeny, readiness/liveness

ostatnie eventy związane z Podem

powód, dlaczego Pod nie wystartował



Eventy: co mówi Kubernetes?

```
kubectl get events  
kubectl get events --sort-by=.metadata.creationTimestamp
```

Eventy często mówią wprost: obraz nie istnieje, Secret nie znaleziony, probe nie przechodzi.

```
Warning   Failed   Error: secret "db-secret" not found
```



Logi aplikacji

```
kubectl logs <pod-name>  
kubectl logs deployment/<deployment-name>  
kubectl logs -f <pod-name>
```

Logi odpowiadają na pytanie: co mówi sama aplikacja?

- błędy startu aplikacji
- brak konfiguracji
- problemy z połączeniem
- informacje diagnostyczne



Poprzedni kontener

```
kubectl logs <pod-name> --previous
```

Przy restartach kontenera aktualne logi mogą być puste.
Wtedy sprawdzamy logi poprzedniej próby uruchomienia.



exec: wejście do kontenera

```
kubectl exec -it <pod-name> -- sh  
kubectl exec -it <pod-name> -- bash
```

sprawdzenie plików konfiguracyjnych

test DNS z wnętrza Poda

sprawdzenie zmiennych środowiskowych

szybka diagnostyka bez przebudowy obrazu



Aplikacja działa, ale nie odpowiada

```
kubectl get svc  
kubectl describe svc <service-name>  
kubectl get endpoints
```

Jeśli Service nie ma endpointów, zwykle selector nie pasuje do labeli Poda.

Service selector



Pod labels



Port-forward jako test

```
kubectl port-forward svc/qa-web 8080:80  
curl http://localhost:8080
```

To szybki sposób sprawdzenia aplikacji bez Ingressa, LoadBalancera i DNS.



Workflow diagnostyczny QA

```
kubectl get pods  
kubectl describe pod <pod-name>  
kubectl logs <pod-name>  
kubectl get events --sort-by=.metadata.creationTimestamp  
kubectl get svc  
kubectl get endpoints  
kubectl port-forward svc/<service-name> 8080:80
```

Ten zestaw komend wystarczy do większości pierwszych analiz problemów aplikacyjnych.



Mini lab: diagnoza krok po kroku

uruchamiamy aplikację nginx

sprawdzamy Pody, Service i endpointy

generujemy ruch przez port-forward

czytamy logi nginx

usuwamy Poda i obserwujemy self-healing

```
kubectl delete pod <pod-name>  
kubectl get pods
```



Do zapamiętania

get

describe

logs

events

Najpierw status, potem szczegóły, potem logi.
Dopiero później zmieniamy konfigurację.

